Data Structures (ETCCDS202)

Unit–3 Assignment: Sorting Algorithms

Name of the School: School of Engineering & Technology

Program : B.Tech (CSE / AIML)

Section : A

Student Name : Disha Saini

Roll No. : 2501730318

Submitted To : Ms. Neetu

1. Introduction

This assignment focuses on implementing and analyzing three sorting algorithms: Insertion Sort, Merge Sort, and Quick Sort. The aim is to study their performance on different types of input data and compare the observed results with their theoretical time complexities.

Sorting is a fundamental concept in data structures, and this assignment helps in understanding how different algorithms behave under varying conditions.

2. Algorithms Implemented

2.1 Insertion Sort

Insertion Sort works by taking one element at a time and inserting it into its correct position in the sorted portion of the list.

- Time Complexity: $O(n^2)$
- Stable: Yes
- In-place: Yes

2.2 Merge Sort

Merge Sort follows a divide-and-conquer approach where the array is divided into smaller parts, sorted individually, and then merged.

- Time Complexity: O(nlogn)
- Stable: Yes
- In-place: No (uses extra memory)

2.3 Quick Sort

Quick Sort selects a pivot element and partitions the array such that smaller elements are on one side and larger elements on the other.

- Time Complexity: Average O(nlogn), Worst $O(n^2)$
- Stable: No
- In-place: Yes

3. Dataset Description

To evaluate the performance of the sorting algorithms, three different types of datasets were generated. These included random data, already sorted data, and reverse sorted data. The use of different input types helps in understanding how each algorithm behaves under varying conditions, such as best case, average case, and worst-case scenarios.

Each type of dataset was tested using three different input sizes: 1000, 5000 and 10000 elements. This allowed for a clear comparison of how the algorithms scale with increasing input size. In total, nine datasets were used for experimentation, ensuring a comprehensive evaluation of performance across both data type and size.

4. Result

The execution time of each sorting algorithm was measured for different dataset types and sizes. The results are recorded in milliseconds and presented in the table below.

Size	Type	Insertion	Merge	Quick
1000	random	17.44	1.37	0.79
1000	sorted	0.08	0.92	32.45
1000	reverse	33.32	0.97	22.88
5000	random	434.35	8.65	5.56
5000	sorted	0.38	5.64	887.08
5000	reverse	896.42	5.79	610.52
10000	random	1726.04	17.74	10.16
10000	sorted	0.80	11.03	3480.56
10000	reverse	3454.08	11.74	2230.43

From the table, it can be observed that Insertion Sort takes significantly more time as the input size increases, especially for reverse-sorted data, confirming its $O(n^2)$ time complexity. Merge Sort shows consistent performance across all input types, demonstrating its O(nlogn) behaviour. Quick Sort performs efficiently on random data but becomes much slower for sorted and reverse-sorted datasets due to the use of the last element as pivot, which leads to unbalanced partitions and worst-case performance.

5. Analysis

5.1 Random Data

For random datasets, Merge Sort and Quick Sort performed significantly faster than Insertion Sort. This is expected because both have a time complexity of O(nlogn), while Insertion Sort has $O(n^2)$.

5.2 Sorted Data

Insertion Sort performed efficiently on sorted data because it requires minimal shifting, making it close to O(n). Quick Sort showed slower performance due to the use of the last element as pivot, which leads to unbalanced partitions.

5.3 Reverse Sorted Data

Insertion Sort showed worst-case performance due to maximum shifting of elements. Quick Sort also performed slower for similar reasons, while Merge Sort maintained consistent performance.

Sort, Merge Sort, and Quick Sort. The aim is to study their performance on different types of input data and compare the observed results with their theoretical time complexities.

Sorting is a fundamental concept in data structures, and this assignment helps in understanding how different algorithms behave under varying conditions.

6. Theory vs Practical Observation

The observed results align with theoretical expectations:

- $O(n^2)$ algorithms become slower as input size increases
- O(nlogn) algorithms perform efficiently on larger datasets

This highlights the importance of selecting appropriate algorithms based on input conditions.

7. Stability and Memory Usage

Algorithm	Stability	In-place
Insertion Sort	Yes	Yes
Merge Sort	Yes	No
Quick Sort	No	Yes

8. Conclusion

This assignment demonstrated how different sorting algorithms behave under different input conditions. Merge Sort provided consistent performance, Quick Sort was efficient on average but sensitive to pivot choice, and Insertion Sort performed well only for small or nearly sorted datasets.

9. References

- Class notes from Data Structures
- Lecture PPTs
- GeeksforGeeks